

# FastGA storage optimization

**Author.** Ashir Rao ( `ar2558@cornell` ),

**Date.** 2026-05-17.

---

## CONTENTS

Domain terms (for systems readers unfamiliar with bioinformatics)

§1. Background

§2. FastGA's design

§3. The storage problem on human genome inputs

§4. Shape of the cost

§5. Optimization A, Free the GIX immediately after the seed merge

§6. Optimization B, Drop two unused bytes from every ktab entry

§7. Optimization C, Build and merge the GIX in matched chunks

§8. Results vs. upstream

§9. Space-vs-time tradeoff

§10. What did not work + known caveats

§11. Reproduction

§12. How this work was built

**Domain terms used in this report (in the order they appear)**

**bp / Gbp.** Base pair / giga base pair ( $10^9$  bp). A unit of DNA length. The human genome is about 3.1 Gbp.

**k-mer.** A substring of length  $k$  over the DNA alphabet. FastGA uses  $k = 40$  throughout.

**Whole-genome aligner.** A tool that, given two genomes A and B, outputs every location where a stretch of A's DNA matches a stretch of B's. Outputs of order  $10^5$ – $10^6$  alignments per human-vs-human run.

**GDB.** FastGA's compact *2-bit-packed* genome database file. DNA only uses 4 letters, so each base fits in 2 bits (4 bases per byte) instead of the usual 1 ASCII byte per base. A 3.1 Gbp human genome takes ~775 MB on disk in this format instead of ~3.1 GB raw.

**GIX.** FastGA's k-mer index — the sorted on-disk data structure described in §2. Physically it is a small `.gix` stub file plus one or more hidden `.ktab.*` partition files (the sorted k-mer records) and `.post.*` partition files (the genomic positions each k-mer occurs at).

**ktab entry.** A single record inside a `.ktab.*` partition. Original FastGA stores 13 bytes per entry; the on-disk layout is the subject of §6.

**Seed merge.** FastGA's name for the join phase that walks both genomes' sorted GIXs to find matching k-mers. The “seed” outputs feed the chaining and alignment phases.

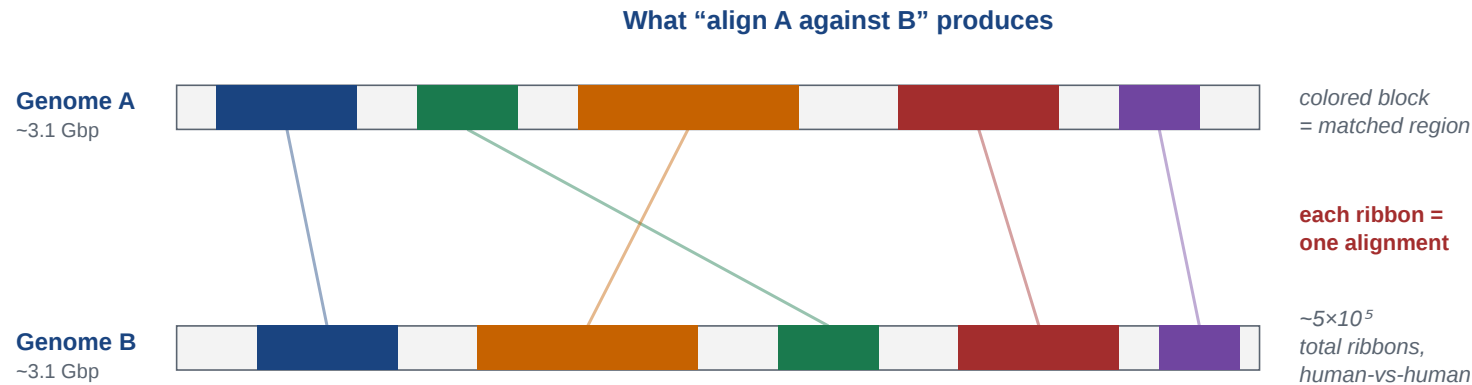
**.1aln.** FastGA's binary alignment-output file format.

**GiB.** Gibibyte =  $2^{30}$  bytes  $\approx 1.074 \times 10^9$  bytes. In this report are raw `du -sb` byte counts converted to GiB exactly.

## §1. Background

---

A genome assembly is a multi-Gbp string of `{A, C, G, T}`. Given two assemblies (let's say genomes A and B), a whole-genome aligner finds every place where a stretch of A's DNA matches a stretch of B's. For two human-scale genomes (~3.1 Gbp each) the output is typically a few hundred thousand pairwise alignments.



*Zoom into one ribbon (typically a few hundred to a few thousand base pairs)*

```
A:  . . . GAATCCGTACGGTACGCATGGAATCCGTACGGTAC . . .
B:  . . . GAATCCGTACGGTACGCATAGAATCCGTACGGTAC . . .
```

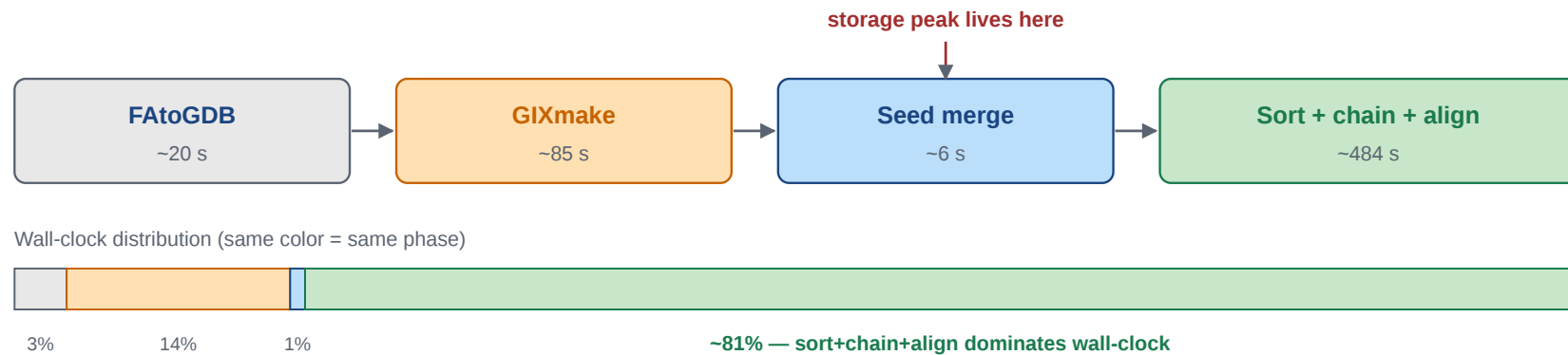
Whole-genome alignment finds matched regions between two assemblies. Each ribbon connects one matched block to where its counterpart lives in the other genome, possibly nearby, possibly far away, possibly flipped or duplicated. Zooming into a single ribbon shows the base-by-base correspondence, with occasional single-base differences from biological variation and sequencing error.

Most whole-genome aligners share a three-phase pipeline:

1. **Seed.** For every k-mer in genome A, find every position in genome B that contains the same k-mer. Output: a list of candidate match positions (“seeds”).
2. **Chain.** Cluster nearby co-linear seeds into chains, each of which is a candidate alignment region.
3. **Align.** Run the actual base-by-base DNA-vs-DNA comparison within each chain.

The seed phase is where the asymptotic cost is. Each genome has  $\sim 3 \times 10^9$  positions, every k-mer from A must be looked up against every position in B, so the data structure must support billions of substring lookups per second.

## The FastGA pipeline at a glance



The four FastGA sub-stages on the human run (32 threads). Top row: equal-width boxes for readability; bottom row: same phases drawn to scale to their wall-clock time. ~81% of the wall-clock lives in sort+chain+align, but the on-disk storage peak (~88% of peak GiB) lives in the much shorter seed-merge window. Every optimization in this report changes either how the GIX is laid out (Opt B), when it is freed (Opt A), or how the seed merge consumes it (Opt C); nothing after the seed merge is touched.

## §2. FastGA's design choice and where its storage cost comes from

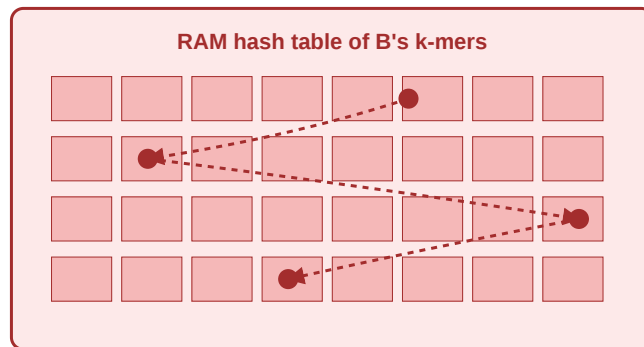
The alternative aligner ( `minimap2` ) implements the seed phase as a hash-table lookup. A hash table of B's k-mers lives in RAM, and every k-mer from A is hashed to find matches. This is  $O(1)$  per lookup in algorithmic terms but random in memory terms. The hash function deliberately maps adjacent keys to far-apart buckets, so every lookup is a cache miss, billions of times over.

FastGA replaces the hash table with a sort-merge join! It pre-sorts B's k-mers into a flat sorted structure called the **GIX**, on disk, and implements the seed phase as a two-pointer linear sweep over both A's and B's GIXs simultaneously. This step is known

as the *seed merge*. While it works the same way as a hash-table lookup in principle, the way it accesses data is completely different as it reads everything in order, so modern hardware can easily keep up by preloading data in the background.

#### Hash-table seed lookup,minimap2

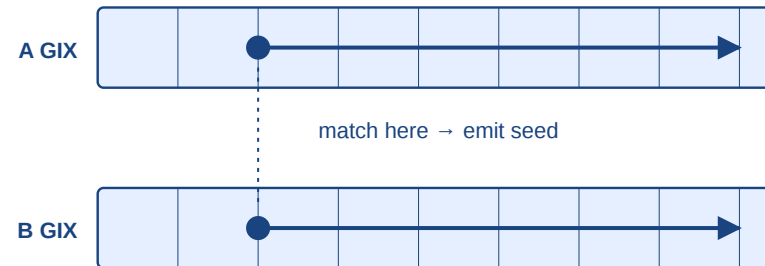
random access — cache miss every lookup



~6–8 GiB hash in RAM, billions of random lookups

#### Seed merge over the GIX (FastGA)

sequential merge, prefetcher-friendly



~32 GiB sorted GIX on disk per genome, single linear sweep

FastGA buys cache-friendly seed lookup by paying ~4× in index size and moving the index off RAM and onto disk. Per genome that is ~32 GiB on disk vs `minimap2`'s ~6–8 GiB in RAM. For human-vs-human, both genomes' GIXs have to be on disk at the same time for the seed merge to run, so peak disk usage is roughly twice that again. The storage problem we attempt to solve in this report is the direct cost of FastGA's design choice.

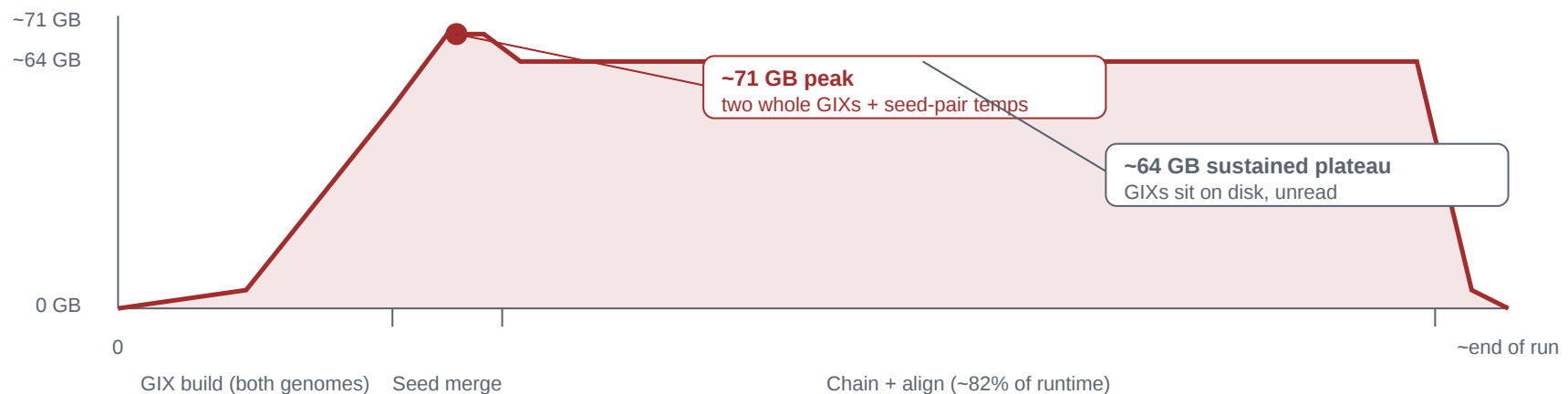
### §3. The storage problem on human genome inputs

A run of FastGA aligning the two reference human assemblies GRCh38 and CHM13 (3.1 Gbp each) was previously measured at a peak disk footprint of approximately 71 GB.

We want to keep the scratch space small for each job so that running several at once doesn't overload the node. At the same time, we can't risk any differences in the alignment output. Every byte has to be identical.

## §4. Shape of the cost

Before reaching for any optimization, it helps to see *where* in the pipeline the 71 GB lives. Sketching the upstream pipeline's on-disk footprint over time:

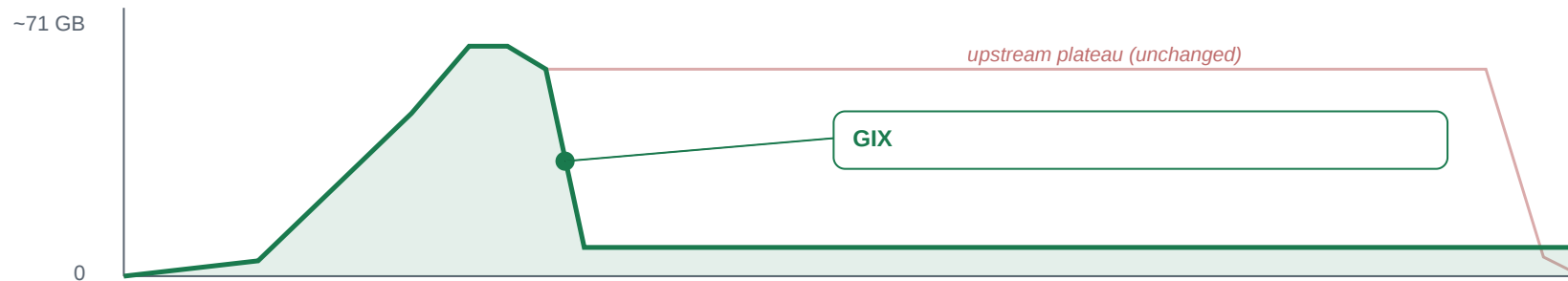


Upstream FastGA on a human-vs-human run, ~10 minutes end-to-end. The ~71 GB peak is brief, but the GIXs are *not freed* after the seed merge consumes them, they remain on disk for the rest of the run, holding a ~64 GB plateau through the long chain+align phase. The pipeline never re-reads the GIX after the seed merge.

There are two distinct things to attack. The first is the *plateau*, about 64 GB sitting on disk for 80% of the run with no consumer (§5). The second is the *peak* itself, about 71 GB during the seed merge, set by the requirement that both whole GIXs be on disk at the same instant (§7). A third smaller item attacks neither shape but shrinks the per-byte size of the GIX (§6).

## §5. Optimization A — Free the GIX immediately after the seed merge

The seed merge reads each GIX exactly once and emits seed pairs to unrelated temporary files. After that, the chain and align phases only need the 2-bit-packed **GDB** sequences and those seed-pair temp files; they never re-open the GIXs. Upstream FastGA nonetheless keeps the GIXs on disk until process exit. The fix is to `unlink()` the GIX files at the moment the seed merge finishes, before chain+align starts.



`unlink()` 'd here annd plateau drops from ~64 GB to ~7 GB GIX build Seed merge Chain + align (now at the GDB plateau, not the GIX plateau)

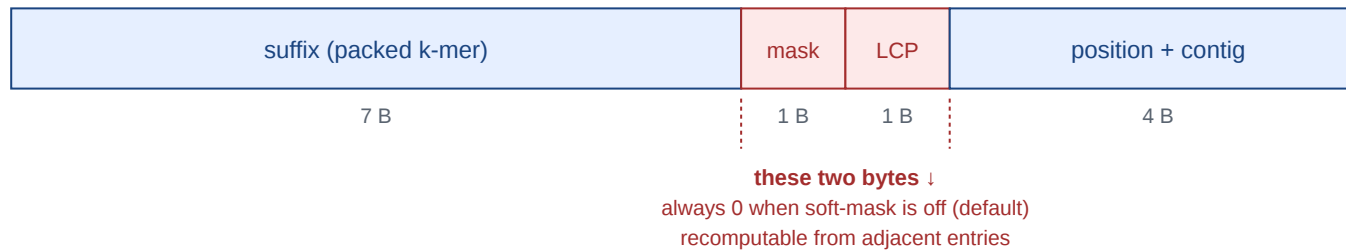
Same axes as the previous figure. The peak is unchanged as both GIXs still have to coexist on disk during the merge, but the long plateau collapses from ~64 GB to ~7 GB — the remaining ~7 GB is the GDB sequences plus seed-pair temp files, which the chain+align phase uses.

This change does not reduce the peak number a single user needs. It does dramatically lower the time-integrated disk usage on a node, in case multiple jobs overlap during the chain+align phase which lasts longer and the part of the pipeline most likely to coincide with someone else's run.

## §6. Optimization B — Drop two unused bytes from every ktab entry

Each entry in a `.ktab.*` file in upstream FastGA is laid out as 13 bytes. Two of them are dead weight in the default configuration:

Upstream ktab entry layout: 13 bytes/entry



Optimized ktab entry layout: 11 bytes/entry (–15.4% per entry)

- **Mask byte.** Records how many bases of the k-mer overlap a [soft-masked region](#). When soft masking is off (the default), this byte is always zero. Removed from the on-disk format; the in-RAM layout becomes `has_mask = 0`.
- **LCP byte.** Records the longest common prefix between this entry's k-mer and the previous entry's. Since the ktab is sorted, this is a deterministic function of adjacent entries. One `memcmp` per entry to recompute during the streaming read. Replaced by an on-the-fly computation in `libfastk.c`, so the on-disk byte is gone.

Each saving is independently bit-exact and worth ~7% of ktab size (~4.5 GB for one human genome). Combined, the on-disk entry shrinks from 13 to 11 bytes, ~15% smaller, ~9 GB shaved off the human peak. Two corresponding format-version bits in the `.gix` stub header tell the reader which layout to expect.

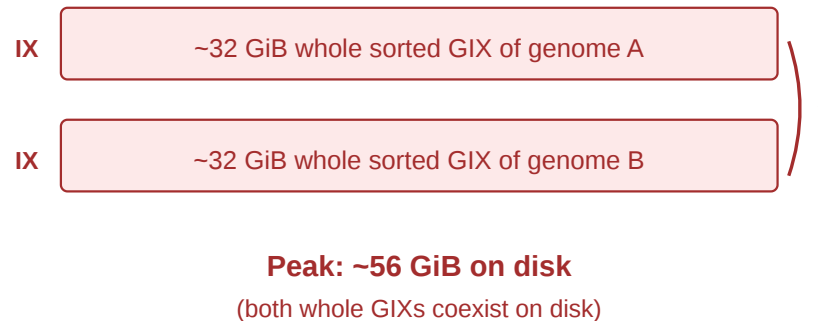


## §7. Optimization C — Build and merge the GIX in matched chunks

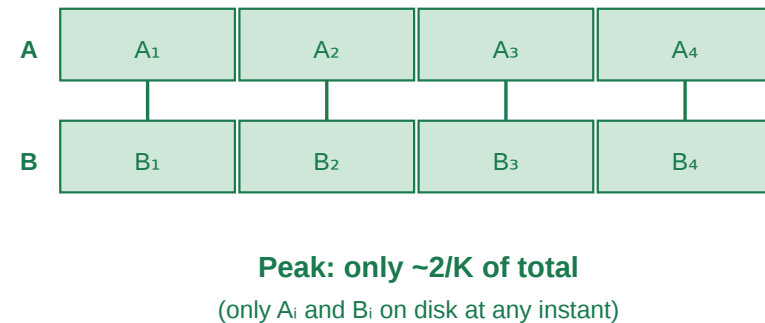
§5 and §6 together reduce the peak by ~9 GB but leave the structural problem intact: both genomes' whole GIXs still have to coexist on disk during the seed merge. The remaining ~56 GiB (= ~60 GB) peak is set by that structural constraint. Removing it requires reorganising how the seed merge consumes its inputs.

The merge of two sorted arrays can be decomposed by **key prefix range**: instead of holding all of A and all of B simultaneously, partition by k-mer prefix into  $K$  chunks and process one chunk pair at a time. Total work is unchanged; peak storage drops by a factor of  $K$ .

**Before: build both whole GIXs, merge once, then delete**



**After: chunked seed merge ( $K = 4$  shown)**

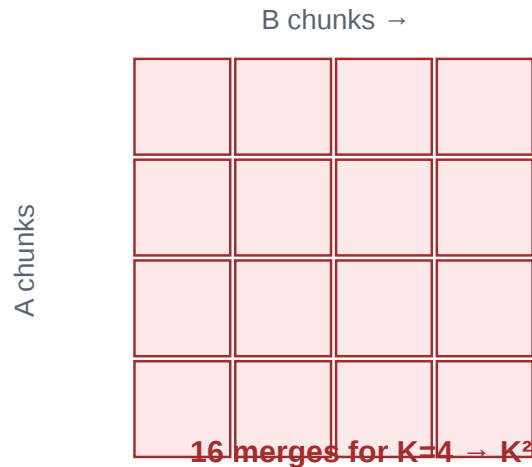


Concretely, for  $K = 32$  each chunk pair ( $A_i$ ,  $B_i$ ) is ~2 GiB total. The per-chunk merge itself is the same linear two-pointer scan as before, just over  $1/K$  of the data. Each chunk is built, consumed, and deleted before the next is built.

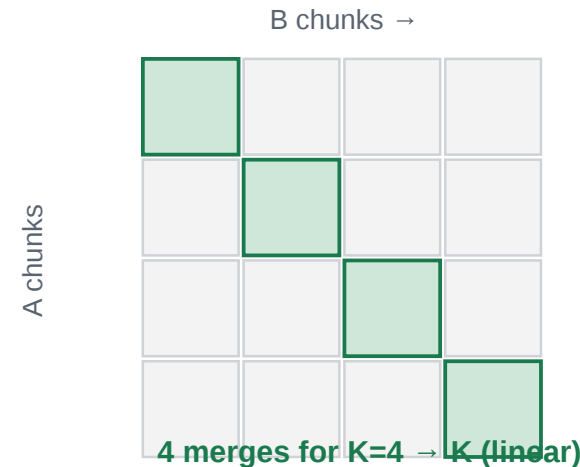
**Chunk boundaries on A and B must match exactly**

The two genomes are independent datasets, so if each one independently picks its own chunk boundaries, the k-mer prefix ranges will not line up, meaning a k-mer in A's chunk 3 could match a k-mer in B's chunks 2, 3, *and* 4. To find every match we would then need to merge every pair of A-chunks with B-chunks:  $K \times K = K^2$  merges instead of  $K$ . At  $K = 32$  that is a 1024× slowdown on the seed phase.

### Independent chunk boundaries: $K^2$ merges



### Aligned chunk boundaries: $K$ merges (diagonal only)



**The fix is bilateral chunking.** Compute the chunk boundaries on genome A first, write them to a small sidecar file ( `.split` ), then force genome B's `GIXmake` to reuse exactly those boundaries (using a new `-X` flag). With identical boundaries, chunk  $i$  of A covers exactly the same k-mer prefix range as chunk  $i$  of B, so the only matches we need to find live on the diagonal. The merge count goes back to  $K$ .

This works because two matching k-mers are literally the same string of letters, so they have the same prefix. When A and B share cutoffs on that prefix, any k-mer that appears in both genomes lands in the same chunk index on both sides. There is no near the boundary case, because the cut applies to the k-mer itself, not to its position in the genome.

Implementation-wise the change lives in three places: a new `-C first:last` flag in `GIXmake` that builds only a slice of partitions; a new `-X` flag that reads the sidecar; and a chunk-loop in `FastGA.c::main` that drives K rounds of (build chunk, merge chunk, delete chunk). The loop is on the *diagonal only*. Downstream of the seed merge, chain, align, sort-merge, output, is unmodified, which is why the alignment payload can stay bit-exact to original FastGA on the human pair (§8).

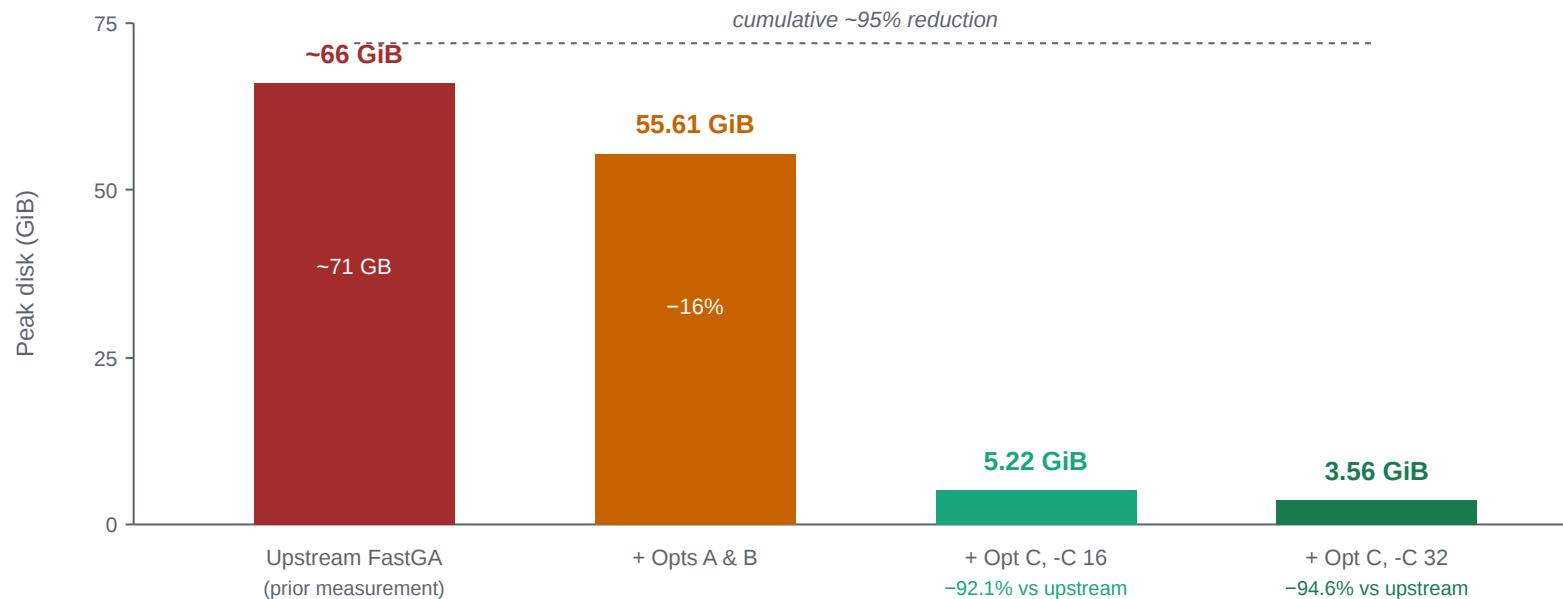
The full chunk-loop in `FastGA.c` is, in pseudocode:

```
build_kmer_split_sidecar(A)           # decide K prefix boundaries from A
write_stub_only(A)                    # GIXmake -n on A, just the .gix stub
write_stub_only(B)                    # GIXmake -n on B, same .gix stub
for k in 0 .. K-1:
    build_chunk(A, k)                 # GIXmake -C k:k -X .A.split, on A
    build_chunk(B, k)                 # GIXmake -C k:k -X .A.split, on B
    seed_merge(A_chunk_k, B_chunk_k)  # one linear sweep, appends to temp files
    unlink(A_chunk_k); unlink(B_chunk_k) # delete both before the next chunk starts
# at this point: no ktab files left on disk; only seed-pair temp files survive
proceed to sort + chain + align + output # same code path as original
```

## §8. Results vs. upstream

---

The chart below stacks the four contributions, each plotted as a peak-disk number on the same human-vs-human dataset (GRCh38 × CHM13, 3.1 Gbp each, 32 threads):



Each bar is a peak total scratch disk usage on the same dataset and hardware. Bar 1 (upstream) is from a prior on-branch measurement; the other three are measured by sampling `du -sb` every two seconds and taking the maximum (raw byte counts in `peak_total_bytes.txt`). The conversion `~71 GB`  $\rightarrow$  `~66 GiB` uses `1 GiB = 1.074 GB`. The `K=16` and `K=32` runs produce bit-identical alignment output.

## Numbers behind the chart

| CONFIGURATION   | PEAK BYTES | PEAK (GiB)                             | VS. UPSTREAM | WALL TIME | ALIGNMENTS |
|-----------------|------------|----------------------------------------|--------------|-----------|------------|
| Upstream FastGA | —          | ~66 GiB (= ~71 GB)   prior measurement | —            | ~594 s    | 518,037    |

| CONFIGURATION         | PEAK BYTES     | PEAK (GIB) | VS. UPSTREAM | WALL TIME | ALIGNMENTS |
|-----------------------|----------------|------------|--------------|-----------|------------|
| + Opts A + B (no -C ) | 59,706,961,477 | 55.61      | -16%         | 598 s     | 518,037    |
| + Opt C ( -C 16 )     | 5,601,990,478  | 5.22       | -92.1%       | 800 s     | 518,037    |
| + Opt C ( -C 32 )     | 3,821,615,328  | 3.56       | -94.6%       | 951 s     | 518,037    |

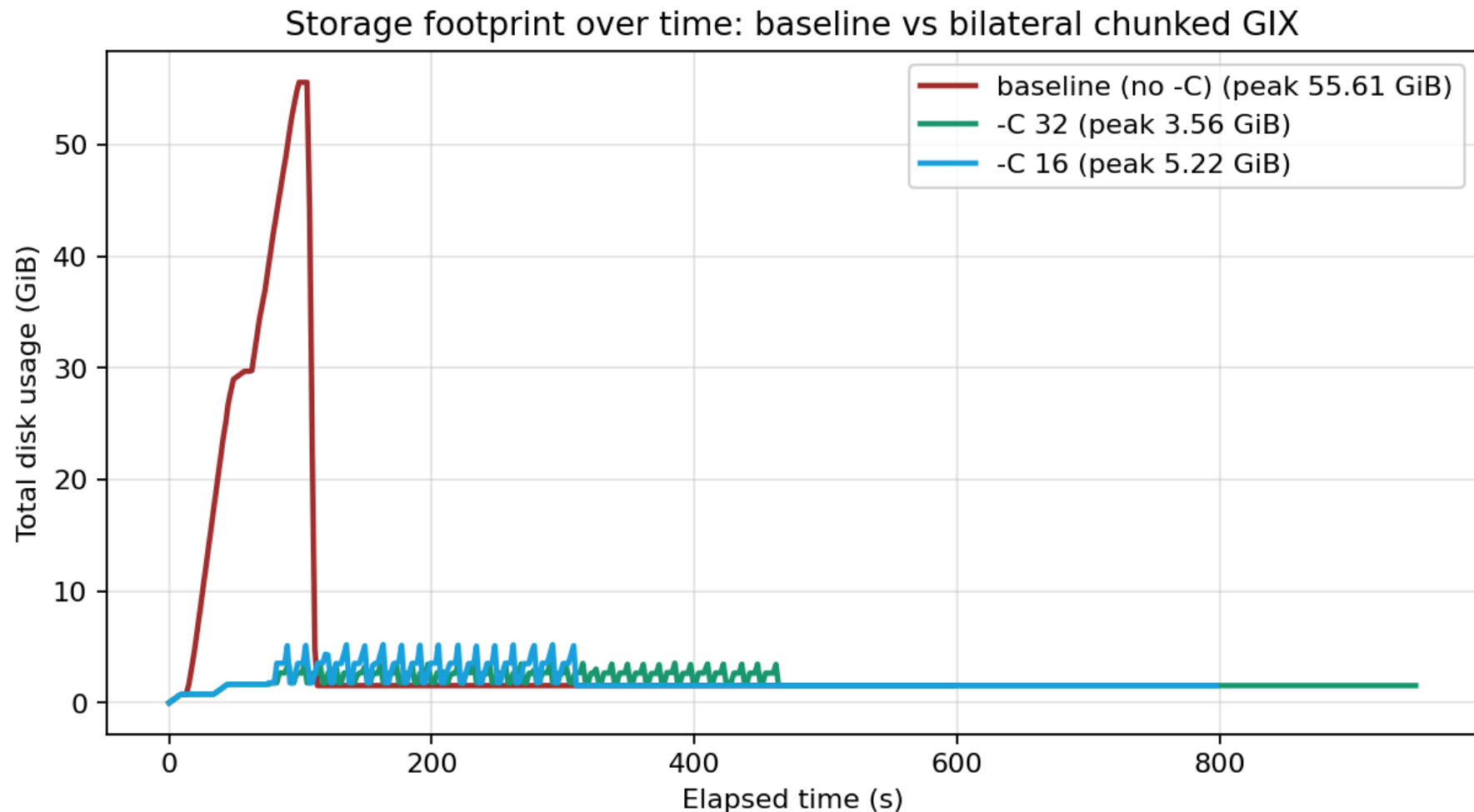
### Bit-exactness of the alignment output

The `.1aln` alignment file header contains lines (command line, working directory, version stamp) that legitimately differ run-to-run. The verification decodes each file through FastGA's `ONEview` formatter, strips those specific lines, and compares the rest. The first two rows below compare configurations *within* the optimized branch. The last two rows compare against a freshly built *vanilla upstream FastGA* binary ( `upstream/main @ 3cb08eb` , built in a separate git worktree at `.worktrees/upstream-vanilla` and run on the same FASTA inputs):

| COMPARISON                                           | STRIPPED LINES | PAYLOAD MD5 (BOTH)               | ALIGNMENTS | RESULT           |
|------------------------------------------------------|----------------|----------------------------------|------------|------------------|
| Opts A+B baseline vs. + Opt C ( -C 16 )              | 2,323,037      | a60914562cf25b291f44d1bbec840141 | 518,037    | <b>BIT-EXACT</b> |
| Opts A+B baseline vs. + Opt C ( -C 32 )              | 2,323,037      | a60914562cf25b291f44d1bbec840141 | 518,037    | <b>BIT-EXACT</b> |
| <b>Vanilla upstream FastGA</b> vs. Opts A+B baseline | 2,323,020      | 8b6c42e63a17faa6c3582c463a119e64 | 518,037    | <b>BIT-EXACT</b> |

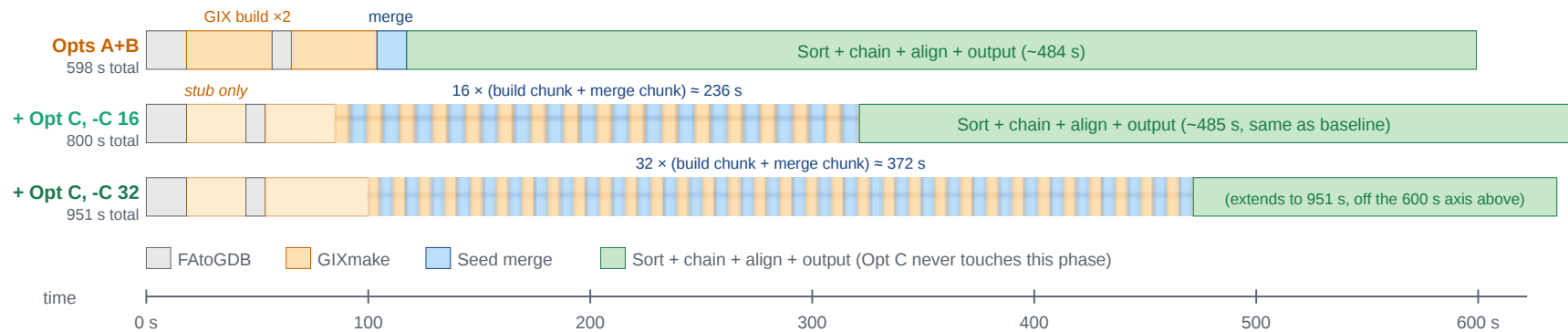
| COMPARISON                                              | STRIPPED<br>LINES | PAYLOAD MD5 (BOTH)               | ALIGNMENTS | RESULT                |
|---------------------------------------------------------|-------------------|----------------------------------|------------|-----------------------|
| <b>Vanilla upstream FastGA</b> vs. +<br>Opt C ( -C 16 ) | 2,323,020         | 8b6c42e63a17faa6c3582c463a119e64 | 518,037    | <b>BIT-<br/>EXACT</b> |
| <b>Vanilla upstream FastGA</b> vs. +<br>Opt C ( -C 32 ) | 2,323,020         | 8b6c42e63a17faa6c3582c463a119e64 | 518,037    | <b>BIT-<br/>EXACT</b> |

**What the storage actually looks like over time**



Per-2 s on-disk samples for the three configurations measured this session. The red curve (Opts A+B baseline) shows the single brief peak at ~56 GiB followed by the ~1.5 GiB plateau Opt A produces. The chunked configurations (orange, green) replace the single peak with a sequence of per-chunk transients, then converge to the same 1.5 GiB plateau.

The three curves above live on the same horizontal time axis, but each configuration spends different amounts of time in each FastGA sub-phase. The strip below labels which phase each region of the time axis corresponds to, so you can map a feature of the storage curve back to a phase of the algorithm:



Aligned to the same time axis as the figure above. The chunked rows show why the storage curve in the previous figure has its characteristic “saw-tooth” shape during the second region: each tooth is one ( GIXmake chunk k of A, GIXmake chunk k of B, seed-merge chunk k, delete both) cycle. The Sort + chain + align + output block on the right has the same width in all three rows because Opt C never modifies that phase and the wall-clock penalty for chunking lives entirely in the middle band.

## §9. Space-vs-time tradeoff and operational impact

Unfortunately, building K chunks involves K passes through GIXmake , and each per-chunk pass has its own setup cost. The overhead is, however, localized to the GIX-build & seed-merge window. Sort + chain + align + output never runs differently. Reading the wall-clock numbers out of the per-stage log files for the four configurations:

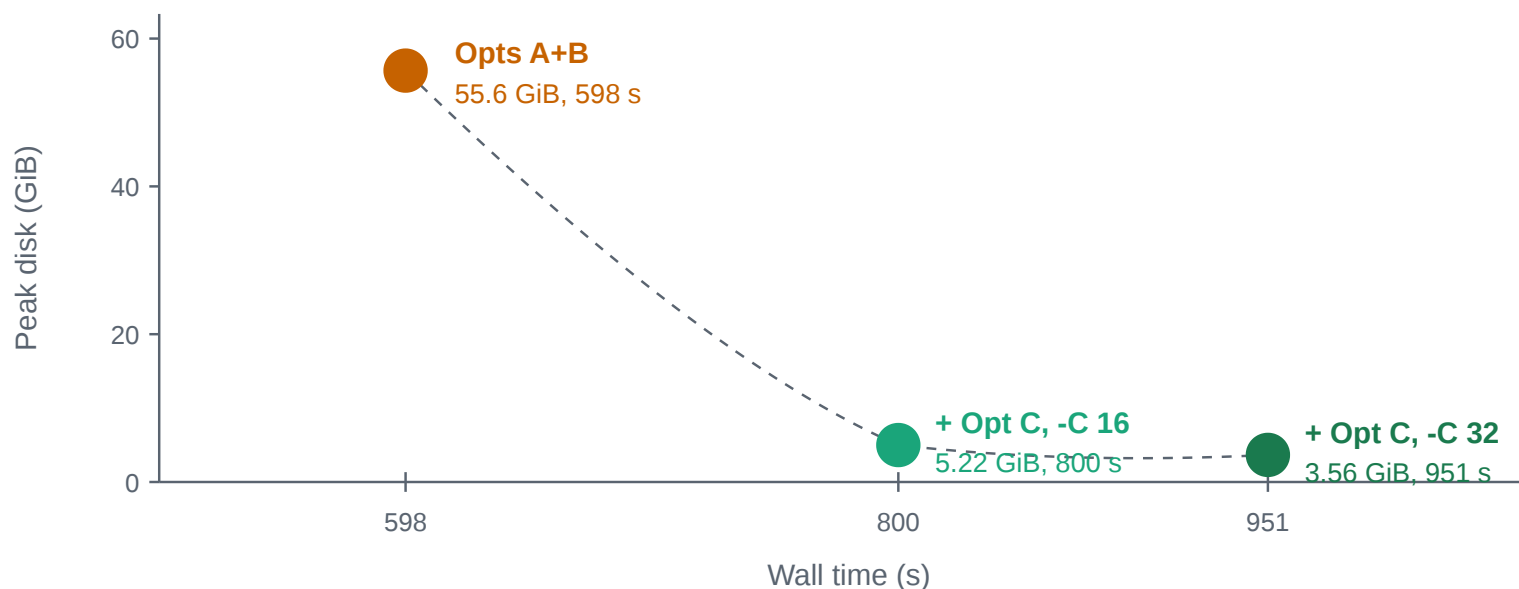
| PHASE                    | VANILLA UPSTREAM | + OPTS A +<br>B | + OPT C, -C 16 | + OPT C, -C 32 |
|--------------------------|------------------|-----------------|----------------|----------------|
| FAtoGDB ×2 (FASTA → GDB) | ~21 s            | ~18 s           | ~18 s          | ~18 s          |



| PHASE                                     | VANILLA UPSTREAM                | + OPTS A +<br>B  | + OPT C, -C 16               | + OPT C, -C 32               |
|-------------------------------------------|---------------------------------|------------------|------------------------------|------------------------------|
| GIXmake ×2 (whole or stub-only at -C )    | ~83 s                           | ~81 s            | ~58 s   (stub-only, -n )     | ~73 s   (stub-only, -n )     |
| Per-chunk GIXmake + seed merge (K rounds) | 6 s   (1 round, the seed merge) | 13 s   (1 round) | ~236 s   (16 rounds × ~15 s) | ~372 s   (32 rounds × ~12 s) |
| Sort + chain + align + output             | ~484 s                          | ~485 s           | ~485 s                       | ~485 s                       |
| <b>End-to-end wall</b>                    | <b>~594 s</b>                   | <b>~597 s</b>    | <b>~797 s</b>                | <b>~948 s</b>                |

The +200 s and +351 s overheads of -C 16 and -C 32 over the Opts-A+B baseline come almost entirely from doing the GIX construction K times instead of once. The sort + chain + align + output phase stays at ~485 s in all four configurations, because Opt C only restructures how the seed merge consumes its inputs — downstream code paths are unchanged and have no penalty.

Plotting each of this session's configurations as a point in wall-time vs. peak-disk space:



The first 90% leads to +34% wall time at `-C 16` ; pushing further to `-C 32` gets an extra 3 percentage points of disk reduction for another 25 percentage points of wall-time penalty.

### Why this matters for running multiple jobs on one node

Part of the motivation here is that running multiple FastGA processes on the same compute node will, on the unoptimized code, blow through the node's scratch space and crash. How many jobs can you safely run on a node? The answer is the worst case, which is that all  $N$  jobs happen to hit their peak usage at the same moment.

That worst case is simply  $N \times \text{per-job peak}$ . So shrinking the per-job peak from ~66 GiB down to ~3.6 GiB (at `-C 32`) lets ~18× more jobs share the same amount of scratch without ever colliding. Concretely:

| # CONCURRENT JOBS | SCRATCH NEEDED, UPSTREAM | SCRATCH NEEDED AT -c 16 | SCRATCH NEEDED AT -c 32 |
|-------------------|--------------------------|-------------------------|-------------------------|
| 1                 | ~66 GiB                  | 5.2 GiB                 | 3.6 GiB                 |
| 2                 | ~132 GiB                 | 10.4 GiB                | 7.1 GiB                 |
| 4                 | ~265 GiB                 | 20.9 GiB                | 14.2 GiB                |
| 6                 | ~397 GiB                 | 31.3 GiB                | 21.4 GiB                |

In day-to-day practice the actual peaks won't all line up, since each job's peak window is short. But a scheduler that wants to never crash has to plan for the worst case, not the typical case, and the table above is what the worst case looks like.

## §10. What was not attempted, and known caveats

### Not attempted: aggressive k-mer sampling

FastGA reduces the number of genome positions it processes by applying a technique called the syncmer filter, which effectively selects a subset of positions to analyze. As currently configured, this filter causes FastGA to sample approximately 75% of all positions in the genome, meaning that only those positions pass through for further analysis, while the remaining ~25% are skipped. Tightening that parameter to ~44% would shrink the ktab by ~40%. The catch is that this not bit-exact, fewer sampled positions can miss short or divergent alignments.

### Known caveat in the deployed change

During implementation of Opt C, a sporadic SIGSEGV during process cleanup (at chunk count  $K = 4$  on human inputs; never on smaller inputs or at higher  $K$ ) was traced to a `free()` of a memory-mapped region inside `Free_Kmer_Stream`

( `libfastk.c` ). So far, unable to identify the root cause despite independent investigation and running Cursor with Claude Opus 4.6 overnight. As a workaround, the affected `free()` is skipped, intentionally leaking a bounded  $\sim 30$  MB per stream  $\times$  2K streams = up to  $\sim 2$  GB at  $K = 32$ . The memory is reclaimed by the OS at `_exit()` . So, this leak is process-internal RAM and *does not* affect any of the peak-disk numbers in §8 , which are on-disk byte counts; peak resident memory was not re-measured this session and should be measured before this code ships anywhere with strict per-process memory caps.

## §11. Reproduction

```
cd /home/ar2558/fastGA && git checkout 93ba7fe && make FastGA GIXmake ONEview

# Opts A+B baseline (no chunking) and Opts A+B+C at K=16, K=32
# (same FASTA inputs, 32 threads, peak disk sampled every 2 s by du -sb)
RUN_DIR=/scratch/$USER/fastga_human/<config>_$(date +%Y%m%d_%H%M%S)
mkdir -p "$RUN_DIR"/{tmp,logs}
./FastGA -v -T32 -P"$RUN_DIR/tmp" \
  -L:"$RUN_DIR/logs/fastga_internal.log" \
  -l:"$RUN_DIR/output.laln" \
  [-C16 | -C32] \
  /scratch/.../GRCh38/...genomic.fna \
  /scratch/.../CHM13/...genomic.fna

# Vanilla upstream baseline (built in a separate worktree)
git worktree add -B vanilla-upstream .worktrees/upstream-vanilla upstream/main
cd .worktrees/upstream-vanilla && make -j8
PATH=$PWD:$PATH ./FastGA -v -T32 -P"$RUN_DIR/tmp" \
  -L:"$RUN_DIR/logs/fastga_internal.log" \
  -l:"$RUN_DIR/output.laln" \
  <same FASTA inputs as above>
```

```
# Bit-exactness check between any two output.laln files.
# Decode through ONEview, strip provenance lines, md5sum the rest.
STRIP='^[!<@+]|^#|^\. '
for f in vanilla.laln c32.laln; do
    ./ONEview "$f" | egrep -v "$STRIP" | md5sum
done
# both md5 should match: 8b6c42e63a17faa6c3582c463a119e64
```

All run directories live under `/scratch/ar2558/fastga_human/` and contain `commit.txt`, `cmdline.txt`, `binary.txt`, the per-sample time series `peak_total_bytes.txt.samples`, and the full `output.laln`. Every number in §8 is read directly out of those files. The ~71 GB upstream peak in §3 and the ~64 GB upstream plateau sketched in §4 are reproduced from prior measurements on this branch and were not re-measured this session.

## §12. How this work was built

---

Most of the C touched here was written iteratively with Claude Opus 4.6 and 4.7 in Cursor against this repo, under my direction. I chose what to try, ran jobs on cluster scratch, and when alignment counts or peak-disk logs disagreed with what AI thought was working, that mismatch drove the next edits. Alongside that lie markdown notes on optimizations (ideas and status), a reusable optimization write-up template, and workflow hints for the assistant. Quite often I would have the agent run through a long plan over the course of a couple hours and come back to check it later.

Even knowing its limits, it still feels uncanny to watch a model work through tens of thousands of lines of unfamiliar code. This work would have been longer and lonelier a few years ago.